

PROJECT DEMONSTRATION

Implement Client Script & UI Policy (Incident)

ServiceNow Incident Form – Dynamic Field Control and Validation

Project Area	ServiceNow / Incident Management
Core Components	UI Policies, UI Policy Actions, Client Scripts, GlideForm
Primary User	Service Desk Agent / Incident User
Purpose	Guide data entry, enforce defined form rules, validate submissions, and reduce incomplete or inconsistent Incident records.

Prepared from the project's six phase materials: Ideation, Requirement Analysis, Project Design, Project Planning, Project Development and Project Documentation.

1. Project Overview

The project addresses inconsistent Incident data entry in ServiceNow. The solution uses UI Policies and Client Scripts to dynamically control form behavior, guide users, handle field values where applicable, validate required information, and prevent invalid or incomplete submissions before the Incident enters downstream management activities.

Problem Being Addressed

- Incident records may be submitted with incomplete, inconsistent, or incorrect information.
- Conditional field requirements may not always be clear to users.
- Manual checking alone may not consistently enforce business rules at the form level.
- Poor data quality can affect triage, routing, reporting and SLA-related work.

Proposed Solution

- Use UI Policies and UI Policy Actions for mandatory, visible/hidden and read-only behavior.
- Use Client Scripts with JavaScript and GlideForm for dynamic client-side behavior.
- Use onSubmit validation to check defined conditions before the record is saved.
- Provide clear feedback so users can correct information before submission.

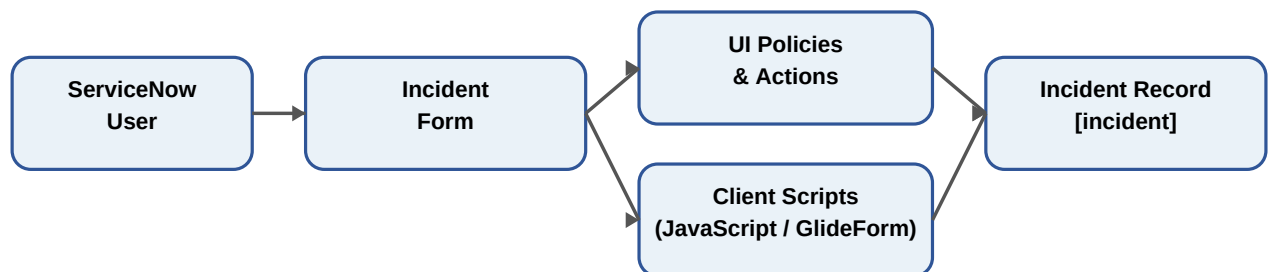
Core objective: make the Incident form guide the user at the moment data is entered and prevent defined invalid submissions.

2. Phase-by-Phase Project Journey

The project documentation is organized into six major phases.

Phase	Purpose	Key Deliverable / Outcome
01 – Ideation	Understand the user and define the problem	Empathy map, problem statements, brainstorming and prioritization.
02 – Requirement Analysis	Understand the user journey and functional requirements	Customer journey, user stories / process and data-flow understanding.
03 – Project Design	Convert requirements into a solution structure	Problem–solution fit, architecture and technology stack.
04 – Project Planning	Plan implementation activities and validation	Implementation sequence, tasks, responsibilities and test approach.
05 – Project Development	Configure and implement the ServiceNow solution	UI Policies, UI Policy Actions and Client Scripts.
06 – Project Documentation	Record the completed work and demonstrate the solution	Final project documentation, results and demonstration evidence.

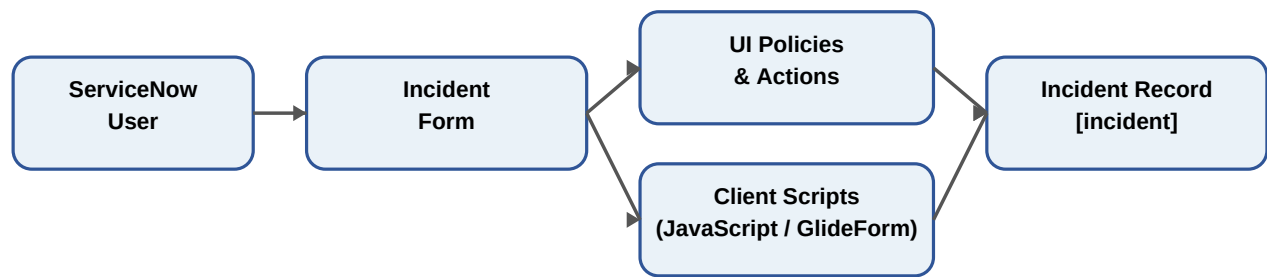
Project Flow



Dynamic field control → validation → consistent Incident submission

3. Technical Architecture

The core implementation runs inside the ServiceNow platform. No external API or machine-learning model is required for the core solution.



Dynamic field control → validation → consistent Incident submission

Technology Stack

Component	Technology / Implementation
User Interface	ServiceNow Web UI / Incident Form
Conditional field behavior	UI Policy / UI Policy Actions
Dynamic client behavior	Client Scripts / JavaScript / GlideForm
Save-time validation	onSubmit Client Script / GlideForm
Data storage	ServiceNow platform data layer / Incident [incident] table
Infrastructure	ServiceNow hosted cloud environment / ServiceNow instance
Security	ServiceNow authentication, roles and ACLs; client scripts/UI policies are for form behavior, not authorization

4. Project Demonstration – Step-by-Step

The following sequence can be used directly as the live demonstration flow during project evaluation.

Step 1 – Open the Incident Form

- Log in to the authorized ServiceNow instance.
- Navigate to the Incident module and open the Incident creation form.
- Observe the initial form fields and default behavior.

Step 2 – Enter Basic Information

- Enter the short description, description and other initial Incident information.
- Point out which fields are required under the configured rules.

Step 3 – Change Controlling Values

- Change relevant values such as Category, Impact, Urgency or Priority.
- Show how the configured UI Policy or Client Script responds to the change.
- Demonstrate a field becoming mandatory, visible/hidden or read-only when its condition is satisfied.

Step 4 – Demonstrate Client-Side Logic

- Show the relevant Client Script configuration and explain its trigger type, such as onLoad, onChange or onSubmit.
- Explain that GlideForm is used to read or set form values and control client-side behavior.

Step 5 – Test Invalid Submission

- Leave a required condition incomplete and click Submit.
- The onSubmit Client Script should display the defined validation feedback and prevent submission when the configured condition is not met.

Step 6 – Complete the Form and Submit

- Provide the missing information and submit again.
- A valid Incident record is created and becomes available for subsequent Incident Management activities.

Demo Video / Recording

<https://drive.google.com/file/d/1Nzg385xJR50eu3NIeGd2wuDMno-IINGi/view>

Screenshots for the demo follow on page 5A.

4A. Demonstration Screenshots

ServiceNow screens used in the demonstration, with a short analysis of what each one shows.

Demo Video / Recording

<https://drive.google.com/file/d/1Nzg385xJR50eu3NleGd2wuDMno-IINGi/view>

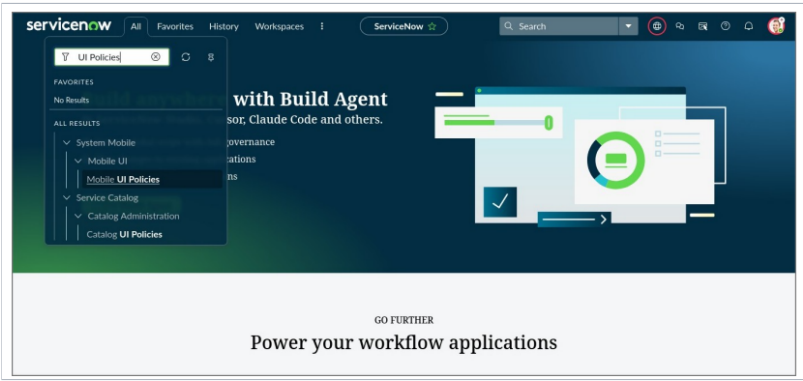


Figure 1: Opening UI Policies from the Application Navigator

Typing **UI Policies** in the navigator filter lists every matching module.

The results visible here are **Mobile UI Policies** (System Mobile) and **Catalog UI Policies** (Service Catalog). These are different from the form UI Policies used on the Incident form.

Tip: for Incident form behavior, open **System UI > UI Policies** (scroll the filtered list).

Short description	Table	Conditions	Reverse if false	On load	Updated	Order
High Impact Control	Incident [incident]	impact=1^EQ	true	true	2026-09-29 07:07:55	100
High Impact Control	Incident [incident]	impact=1^EQ	true	true	2026-09-28 23:55:49	100
Weekly Rotations Not All Day	Roster [cmn_rota_roster]	rotation_interval_type=weekly^rotation_a...	false	true	2026-06-13 03:10:31	100
Weekly Rotations All Day	Roster [cmn_rota_roster]	rotation_interval_type=weekly^rotation_a...	false	true	2026-06-13 03:10:31	100
No Reminders	Roster [cmn_rota_roster]	send_reminders=false^EQ	true	true	2026-06-13 03:10:31	100
Show pricing implications checkbox	Question [question]	type=8^Ortype=21^EQ	true	true	2026-06-13 02:56:32	100
Show price if checked	Question [question]	type=7^EQ	true	true	2026-06-13 02:56:32	100
Show Multiple Choices and Choice price	Item Option [sc_cat_item_option]	type=multiple_choice^Ortype=numeric_scal...	true	true	2026-06-13 02:56:32	100

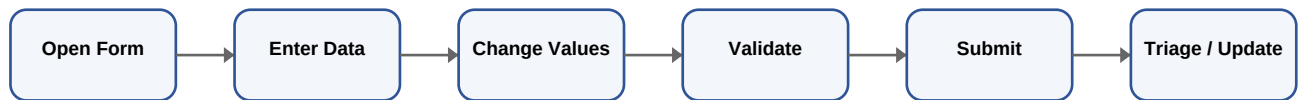
Figure 2: UI Policies list filtered for "high impact"

Policy **High Impact Control** targets **Incident [incident]** with condition **impact=1** (Impact = High).

Reverse if false = true: the effect is undone automatically when Impact is no longer High. **On load = true:** it also applies when the form opens. Order is 100.

Two records share this name (updated 2026-09-28 and 2026-09-29) – confirm one is not a duplicate or leftover version. The list shows 1,708 UI Policies in total; the Roster, Question and Item Option rows are unrelated to Incident.

5. End-to-End Demonstration Flow



Customer journey implemented through UI Policies and Client Scripts

What to explain during the demonstration

- **UI Policy:** Controls form behavior according to a condition, such as making a field mandatory, visible/hidden or read-only.
- **Client Script:** Executes JavaScript on the client side for dynamic form behavior.
- **onChange:** Responds immediately when a selected field value changes.
- **onLoad:** Applies logic when the Incident form loads.
- **onSubmit:** Performs final client-side checks before submission and can stop the save when a defined validation condition fails.
- **GlideForm:** Provides client-side methods for reading and manipulating form fields.

Demonstration narrative

First, we open the Incident form and enter the basic information. When the user changes a controlling value such as Category, Impact or Urgency, the configured UI Policy or Client Script changes the form behavior. The user is guided toward the required information. Before saving, the onSubmit Client Script performs the final client-side validation. If the defined condition is not satisfied, submission is prevented and the user receives feedback. Once the information is corrected, the Incident can be submitted and continues through the normal Incident Management process.

6. Testing & Expected Results

Test Case	Action	Expected Result
TC-01	Open Incident form	Form loads and configured initial behavior is applied.
TC-02	Enter basic Incident details	User can enter the required information.
TC-03	Change a controlling field	Relevant UI Policy / onChange behavior is triggered.
TC-04	Leave a defined required condition incomplete	Validation feedback is shown and submission is prevented when the onSubmit is triggered.
TC-05	Complete the required information	Validation condition is satisfied.
TC-06	Submit a valid Incident	Incident record is created successfully.
TC-07	Update an existing Incident	Configured client-side behavior continues to guide the user during updates.

Evidence to show during the live demo

- Incident form before and after a controlling value changes.
- UI Policy and UI Policy Action configuration.
- Client Script configuration and relevant JavaScript / GlideForm logic.
- Validation message when an invalid submission is attempted.
- Successfully created Incident record after the validation condition is satisfied.

7. Project Outcome

The completed solution is designed to improve the quality and consistency of Incident data at the point of entry.

- Dynamic form behavior based on configured conditions.
- Clearer guidance for Service Desk Agents / Incident Users.
- Client-side validation before submission.
- Reduced dependence on manual checking for defined form-level rules.
- More consistent information for triage, routing, reporting and downstream Incident Management activities.

Important implementation boundary

UI Policies and Client Scripts control client-side form behavior. They should not be treated as a security boundary or as a replacement for ServiceNow authorization, roles or ACLs.

Suggested Live Demo Order

1	Problem	Explain why incomplete or inconsistent Incident data is an issue.
2	Solution	Introduce UI Policies + Client Scripts.
3	Configuration	Show the configured rules and scripts.
4	Behavior	Change a controlling field and show dynamic behavior.
5	Validation	Attempt an invalid submission and show feedback.
6	Success	Correct the data and create the Incident record.

8. Conclusion

Implement Client Script & UI Policy (Incident) demonstrates a practical ServiceNow approach to improving Incident form behavior. The project connects user needs identified during ideation and requirement analysis with a concrete technical design using UI Policies, UI Policy Actions and Client Scripts. The demonstration focuses on dynamic field control, client-side validation and successful Incident submission.

Key Learning Points

- UI Policies are useful for declarative form behavior and conditional field states.
- Client Scripts provide programmable client-side behavior using JavaScript and GlideForm.
- onChange logic can respond immediately to user selections.
- onSubmit validation can stop a defined invalid submission before save.
- Good client-side design should guide users without replacing platform security controls.

Project Demonstration Checklist

- ServiceNow instance accessible
- Incident module/form accessible
- UI Policy configured
- UI Policy Action configured
- Client Script(s) configured
- Validation scenario prepared
- Successful submission scenario prepared
- Screenshots/evidence available for documentation

References / Source Materials

- Project Ideation materials: Empathy Map, Define Problem Statements, Brainstorm & Idea Prioritization.
- Project Requirement Analysis and Customer Journey materials.
- Project Design: Problem–Solution Fit and Technology Stack.
- ServiceNow concepts used in the project: UI Policies, UI Policy Actions, Client Scripts, GlideForm and Incident [incident] table.